Homework 1: Concrete Compressive Strength Regression

在本次作业中，我使用混凝土抗压强度数据集进行分析，并尝试使用多种回归模型对混凝土强度进行预测。

In [31]:
```
# -*- coding: utf-8 -*-

import warnings
warnings.filterwarnings("ignore")

from pathlib import Path
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
```

In [32]:
```
# 0. 全局设置
```

In [33]:
```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Arial Unicode M
plt.rcParams['axes.unicode_minus'] = False

DATA_PATH = Path("Concrete_Data_Yeh.csv")  # 请确保 CSV 和当前 notebook/脚本位于
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

In [34]:
```
# 1. 读取数据
```

In [35]:
```
if not DATA_PATH.exists():
    raise FileNotFoundError(
        f"未找到数据文件：{DATA_PATH}\n"
        "请将 Concrete_Data_Yeh.csv 放在当前 notebook/脚本所在目录中，或修改 DATA
    )

# 原始列名
# ['cement', 'slag', 'flyash', 'water', 'superplasticizer',
#  'coarseaggregate', 'fineaggregate', 'age', 'csMPa']
df = pd.read_csv(DATA_PATH)
```

```python
print("=" * 80)
print("1) 数据集基本信息")
print("=" * 80)
print("数据维度：", df.shape)
print("列名：", list(df.columns))
print("\n前 5 行数据：")
print(df.head())
print("\n缺失值统计：")
print(df.isnull().sum())
print("\n描述性统计：")
print(df.describe())
```

```
================================================================================
1) 数据集基本信息
================================================================================
数据维度： (1030, 9)
列名： ['cement', 'slag', 'flyash', 'water', 'superplasticizer', 'coarseaggregat
e', 'fineaggregate', 'age', 'csMPa']

前 5 行数据：
   cement   slag  flyash  water  superplasticizer  coarseaggregate  \
0   540.0    0.0     0.0  162.0               2.5           1040.0
1   540.0    0.0     0.0  162.0               2.5           1055.0
2   332.5  142.5     0.0  228.0               0.0            932.0
3   332.5  142.5     0.0  228.0               0.0            932.0
4   198.6  132.4     0.0  192.0               0.0            978.4

   fineaggregate  age  csMPa
0          676.0   28  79.99
1          676.0   28  61.89
2          594.0  270  40.27
3          594.0  365  41.05
4          825.5  360  44.30

缺失值统计：
cement              0
slag                0
flyash              0
water               0
superplasticizer    0
coarseaggregate     0
fineaggregate       0
age                 0
csMPa               0
dtype: int64

描述性统计：
            cement         slag       flyash        water  superplasticizer  \
count  1030.000000  1030.000000  1030.000000  1030.000000       1030.000000
mean    281.167864    73.895825    54.188350   181.567282          6.204660
std     104.506364    86.279342    63.997004    21.354219          5.973841
min     102.000000     0.000000     0.000000   121.800000          0.000000
25%     192.375000     0.000000     0.000000   164.900000          0.000000
50%     272.900000    22.000000     0.000000   185.000000          6.400000
75%     350.000000   142.950000   118.300000   192.000000         10.200000
max     540.000000   359.400000   200.100000   247.000000         32.200000

       coarseaggregate  fineaggregate          age        csMPa
count      1030.000000    1030.000000  1030.000000  1030.000000
mean        972.918932     773.580485    45.662136    35.817961
std          77.753954      80.175980    63.169912    16.705742
min         801.000000     594.000000     1.000000     2.330000
25%         932.000000     730.950000     7.000000    23.710000
50%         968.000000     779.500000    28.000000    34.445000
75%        1029.400000     824.000000    56.000000    46.135000
max        1145.000000     992.600000   365.000000    82.600000
```

In [36]:
```
# 2. 特征与标签划分
```

In [37]:
```
feature_cols = [
    'cement', 'slag', 'flyash', 'water', 'superplasticizer',
    'coarseaggregate', 'fineaggregate', 'age'
```

```
]
target_col = 'csMPa'

X = df[feature_cols].copy()
y = df[target_col].copy()
```

In [38]:
```
# 3. 按要求划分训练集 / 测试集
#    前 80% 训练，后 20% 测试
```

In [39]:
```
train_size = int(len(df) * 0.8)

X_train = X.iloc[:train_size].reset_index(drop=True)
X_test = X.iloc[train_size:].reset_index(drop=True)
y_train = y.iloc[:train_size].reset_index(drop=True)
y_test = y.iloc[train_size:].reset_index(drop=True)

print("\n" + "=" * 80)
print("2) 数据划分")
print("=" * 80)
print(f"训练集大小: {X_train.shape[0]}")
print(f"测试集大小: {X_test.shape[0]}")
```

```
================================================================================
2) 数据划分
================================================================================
训练集大小: 824
测试集大小: 206
```

In [40]:
```
# 4. 数据预处理
#    本数据无缺失值，这里仍保留标准流程
```

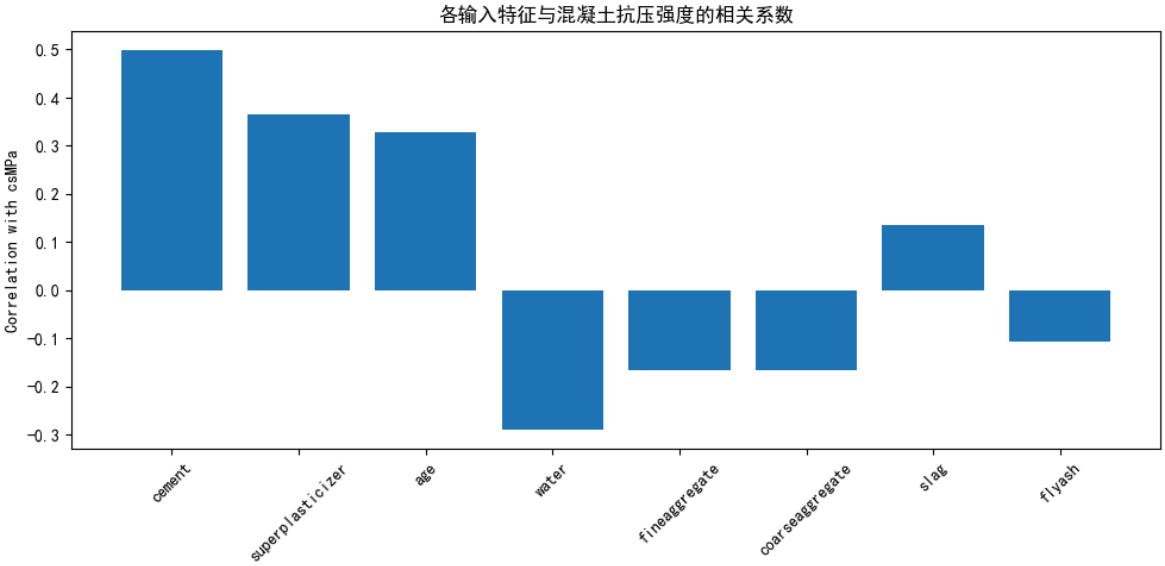
In [41]:
```
# 若存在缺失值，可以使用以下方式:
# X_train = X_train.fillna(X_train.mean(numeric_only=True))
# X_test = X_test.fillna(X_train.mean(numeric_only=True))

# 标准化（仅用训练集拟合，再作用于测试集，避免数据泄露）
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

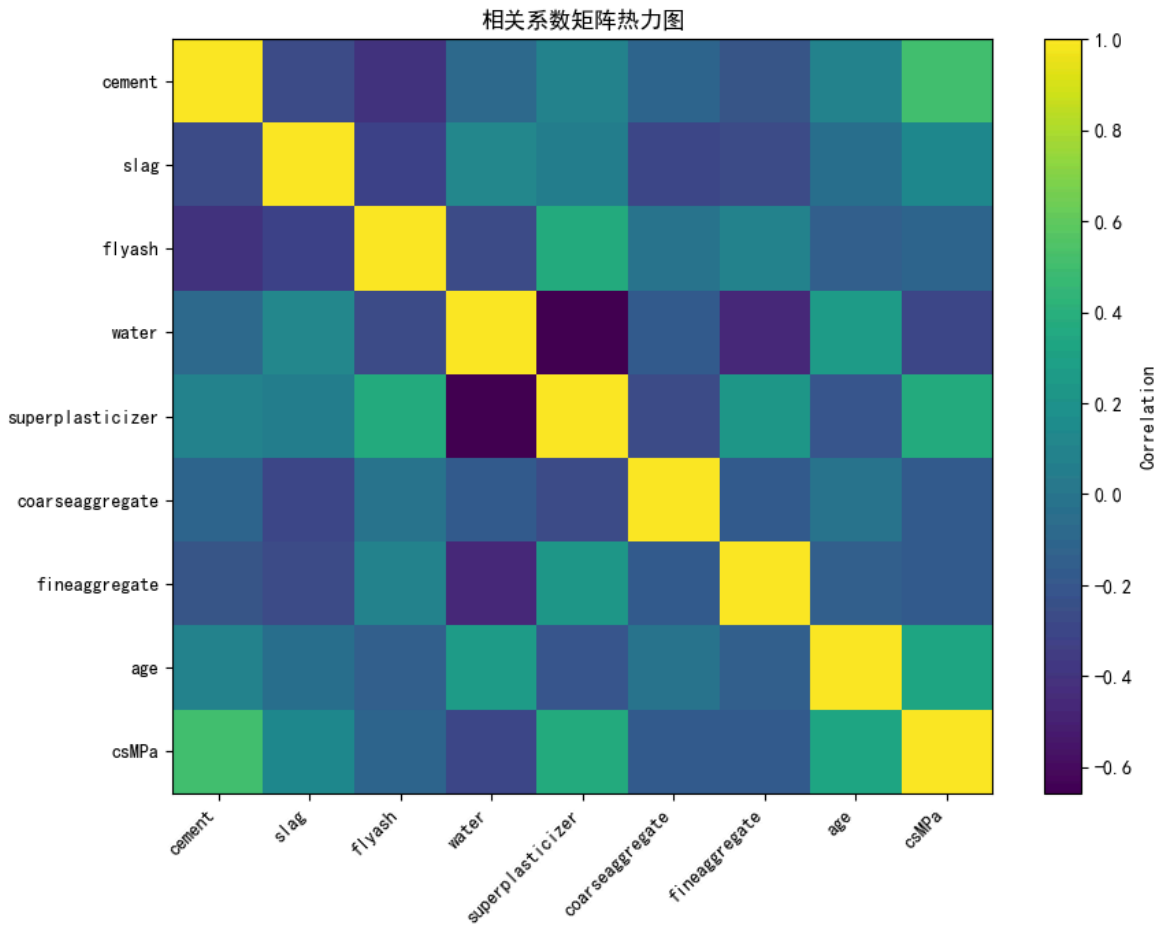
In [42]:
```
# 5. 相关性分析
```

In [43]:
```
print("\n" + "=" * 80)
print("3) 相关性分析")
print("=" * 80)

corr_matrix = df.corr(numeric_only=True)
target_corr = corr_matrix[target_col].drop(target_col).sort_values(key=lambda s:

print("各特征与抗压强度的相关系数（按绝对值降序）: ")
print(target_corr)

# 选择与目标最相关的前 5 个特征（你也可以改成前 4 或前 6）
selected_features = target_corr.index[:5].tolist()
print("\n选取的主要特征（Top 5）: ", selected_features)

X_train_sel = X_train[selected_features].copy()
X_test_sel = X_test[selected_features].copy()
```

```
sel_scaler = StandardScaler()
X_train_sel_scaled = sel_scaler.fit_transform(X_train_sel)
X_test_sel_scaled = sel_scaler.transform(X_test_sel)

# 绘制特征与目标相关性柱状图
plt.figure(figsize=(10, 5))
plt.bar(target_corr.index, target_corr.values)
plt.xticks(rotation=45)
plt.title('各输入特征与混凝土抗压强度的相关系数')
plt.ylabel('Correlation with csMPa')
plt.tight_layout()
plt.show()

# 绘制相关系数矩阵热力图（matplotlib实现，避免额外依赖）
plt.figure(figsize=(9, 7))
plt.imshow(corr_matrix, aspect='auto')
plt.colorbar(label='Correlation')
plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns, rotation=45, ha
plt.yticks(range(len(corr_matrix.columns)), corr_matrix.columns)
plt.title('相关系数矩阵热力图')
plt.tight_layout()
plt.show()
```

```
================================================================================
3) 相关性分析
================================================================================
各特征与抗压强度的相关系数（按绝对值降序）：
cement              0.497832
superplasticizer    0.366079
age                 0.328873
water              -0.289633
fineaggregate      -0.167241
coarseaggregate    -0.164935
slag                0.134829
flyash             -0.105755
Name: csMPa, dtype: float64

选取的主要特征（Top 5）： ['cement', 'superplasticizer', 'age', 'water', 'fineagg
regate']
```



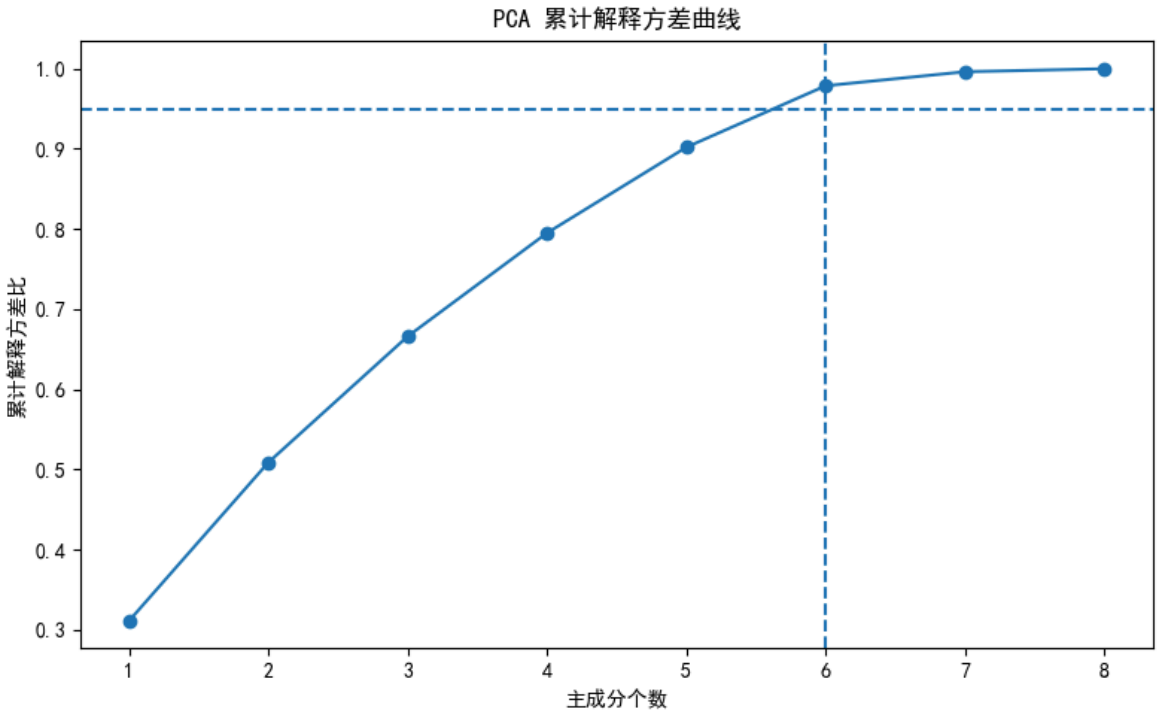


In [44]:
```
# 6. PCA 主成分分析
```

In [45]:
```
print("\n" + "=" * 80)
print("4) PCA 主成分分析")
print("=" * 80)

pca_full = PCA()
pca_full.fit(X_train_scaled)
explained_ratio = pca_full.explained_variance_ratio_
cum_explained_ratio = np.cumsum(explained_ratio)

print("各主成分解释方差比：")
for i, ratio in enumerate(explained_ratio, 1):
    print(f"PC{i}: {ratio:.4f}")

print("\n累计解释方差比：")
for i, ratio in enumerate(cum_explained_ratio, 1):
    print(f"前{i}个主成分: {ratio:.4f}")

# 选择累计解释方差 >= 95% 的主成分个数
n_components_95 = np.argmax(cum_explained_ratio >= 0.95) + 1
print(f"\n达到 95% 累计解释方差所需主成分数：{n_components_95}")

pca = PCA(n_components=n_components_95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

plt.figure(figsize=(8, 5))
plt.plot(range(1, len(cum_explained_ratio) + 1), cum_explained_ratio, marker='o'
plt.axhline(y=0.95, linestyle='--')
plt.axvline(x=n_components_95, linestyle='--')
```

```
plt.xlabel('主成分个数')
plt.ylabel('累计解释方差比')
plt.title('PCA 累计解释方差曲线')
plt.tight_layout()
plt.show()
```

```
================================================================================
4) PCA 主成分分析
================================================================================
各主成分解释方差比：
PC1: 0.3108
PC2: 0.1978
PC3: 0.1574
PC4: 0.1291
PC5: 0.1070
PC6: 0.0767
PC7: 0.0175
PC8: 0.0037

累计解释方差比：
前1个主成分: 0.3108
前2个主成分: 0.5086
前3个主成分: 0.6660
前4个主成分: 0.7950
前5个主成分: 0.9020
前6个主成分: 0.9788
前7个主成分: 0.9963
前8个主成分: 1.0000

达到 95% 累计解释方差所需主成分数: 6
```



In [46]:
```
# 7. 定义评估函数
```

In [47]:
```
def evaluate_regression(y_true, y_pred, model_name="Model"):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
```

```
    print(f"\n[{model_name}] 评估结果")
    print(f"MSE  = {mse:.4f}")
    print(f"RMSE = {rmse:.4f}")
    print(f"MAE  = {mae:.4f}")
    print(f"R²   = {r2:.4f}")

    return {
        "Model": model_name,
        "MSE": mse,
        "RMSE": rmse,
        "MAE": mae,
        "R2": r2,
    }


def plot_prediction(y_true, y_pred, model_name="Model"):
    # 真实值 vs 预测值折线图
    plt.figure(figsize=(12, 5))
    plt.plot(y_true.values, label='True')
    plt.plot(y_pred, label='Predicted')
    plt.title(f'{model_name}: 测试集真实值与预测值对比')
    plt.xlabel('样本编号')
    plt.ylabel('Concrete Strength (MPa)')
    plt.legend()
    plt.tight_layout()
    plt.show()

    # 散点图
    plt.figure(figsize=(6, 6))
    plt.scatter(y_true, y_pred, alpha=0.7)
    min_v = min(np.min(y_true), np.min(y_pred))
    max_v = max(np.max(y_true), np.max(y_pred))
    plt.plot([min_v, max_v], [min_v, max_v], linestyle='--')
    plt.xlabel('True Value')
    plt.ylabel('Predicted Value')
    plt.title(f'{model_name}: 真实值 vs 预测值')
    plt.tight_layout()
    plt.show()
```

In [48]:

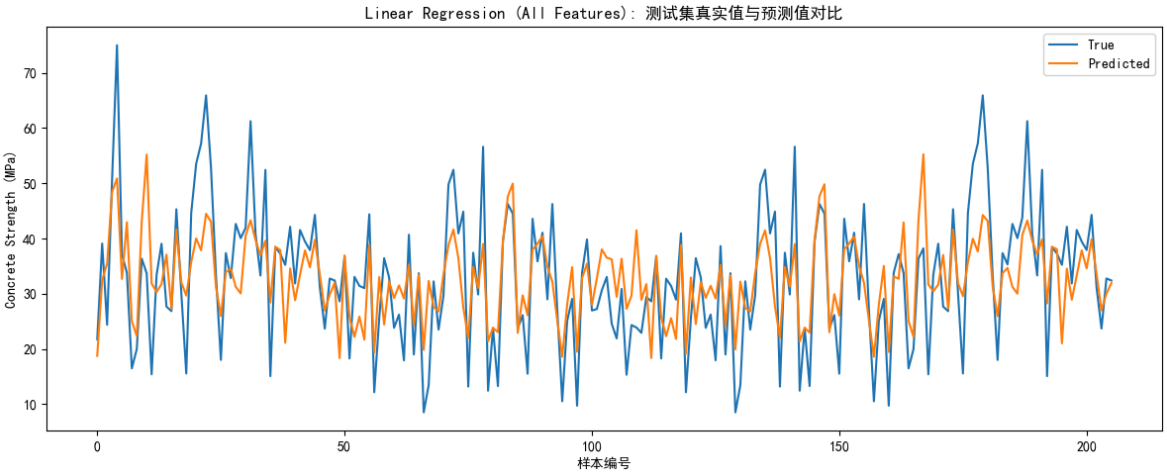
```
# 8. 线性回归模型
```

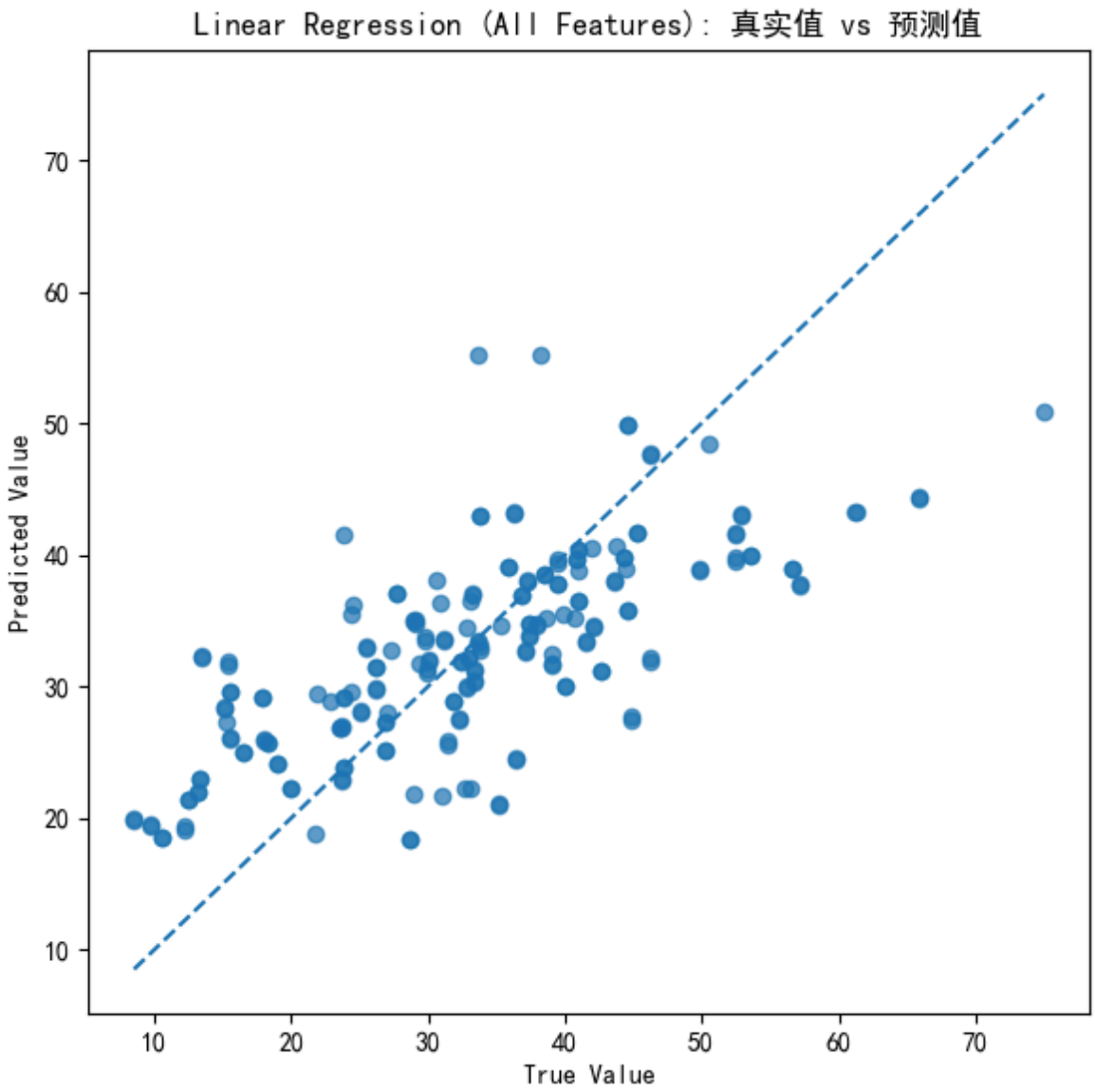
In [49]:

```
print("\n" + "=" * 80)
print("5) 线性回归")
print("=" * 80)

# 8.1 使用全部特征
lr_all = LinearRegression()
lr_all.fit(X_train_scaled, y_train)
y_pred_lr_all = lr_all.predict(X_test_scaled)
res_lr_all = evaluate_regression(y_test, y_pred_lr_all, "Linear Regression (All
plot_prediction(y_test, y_pred_lr_all, "Linear Regression (All Features)")

# 8.2 使用相关性筛选后的特征
lr_sel = LinearRegression()
lr_sel.fit(X_train_sel_scaled, y_train)
y_pred_lr_sel = lr_sel.predict(X_test_sel_scaled)
res_lr_sel = evaluate_regression(y_test, y_pred_lr_sel, "Linear Regression (Sele
plot_prediction(y_test, y_pred_lr_sel, "Linear Regression (Selected Features)")
```

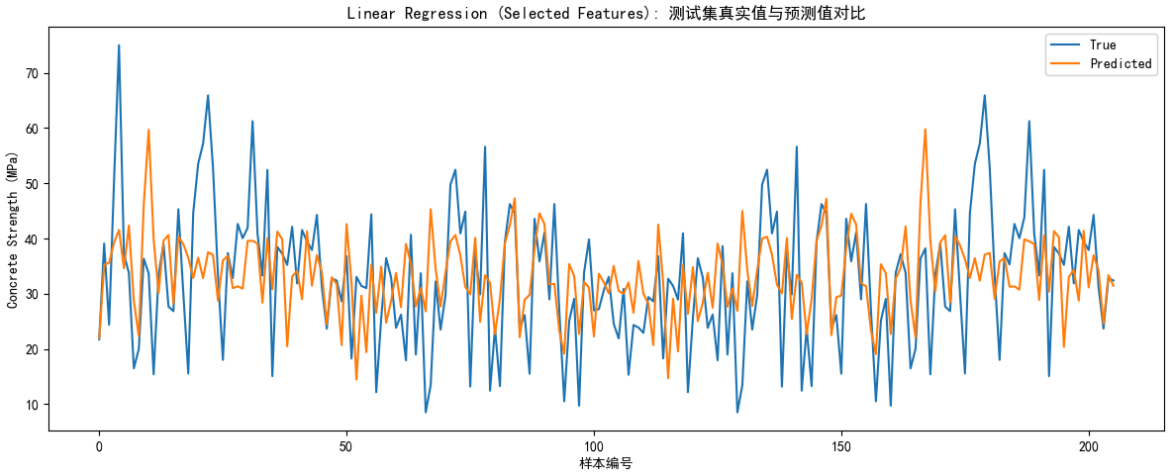
```
# 8.3 使用 PCA 后的特征
lr_pca = LinearRegression()
lr_pca.fit(X_train_pca, y_train)
y_pred_lr_pca = lr_pca.predict(X_test_pca)
res_lr_pca = evaluate_regression(y_test, y_pred_lr_pca, "Linear Regression (PCA
plot_prediction(y_test, y_pred_lr_pca, "Linear Regression (PCA Features)")
```

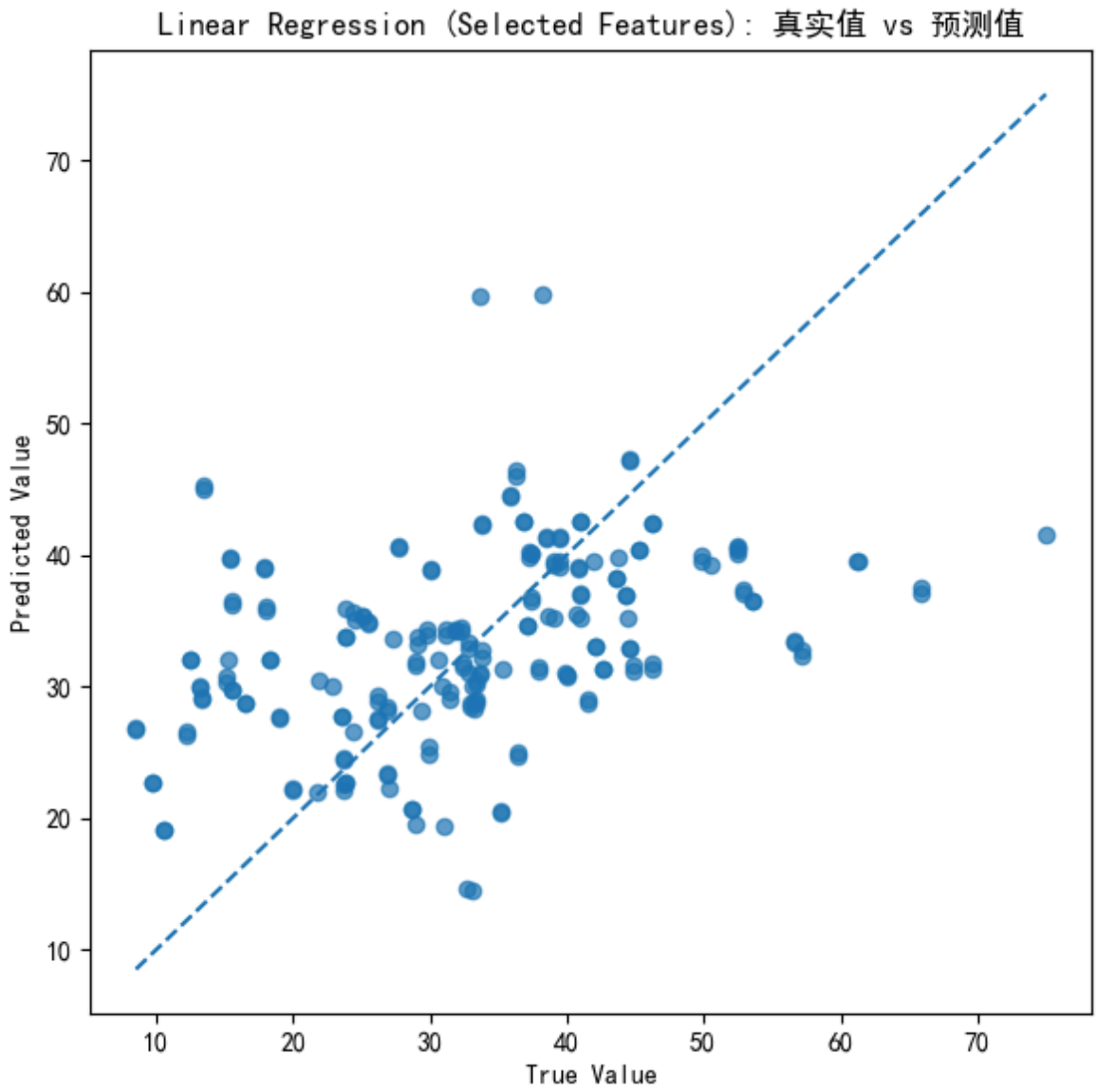
```
================================================================================
5) 线性回归
================================================================================

[Linear Regression (All Features)] 评估结果
MSE  = 75.7439
RMSE = 8.7031
MAE  = 6.8723
R²   = 0.5037
```



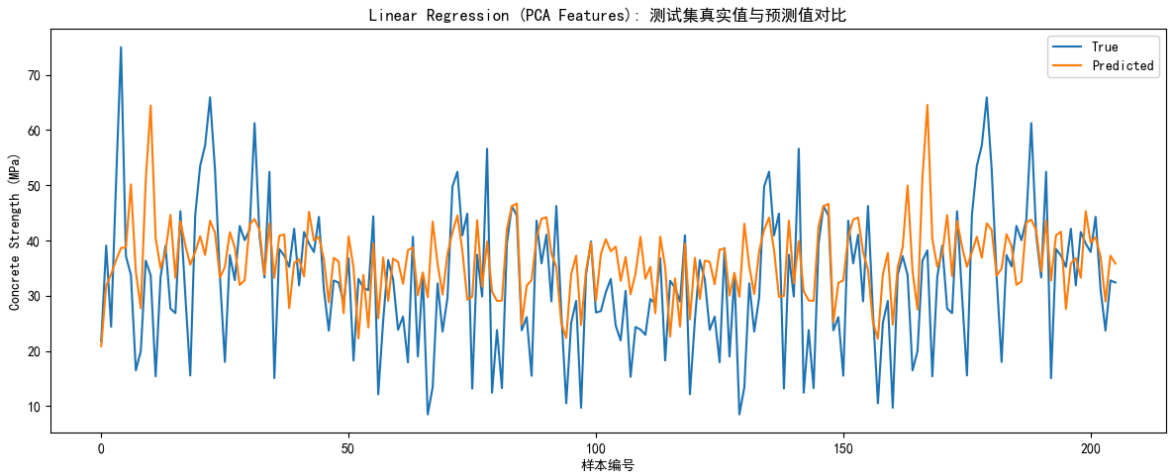


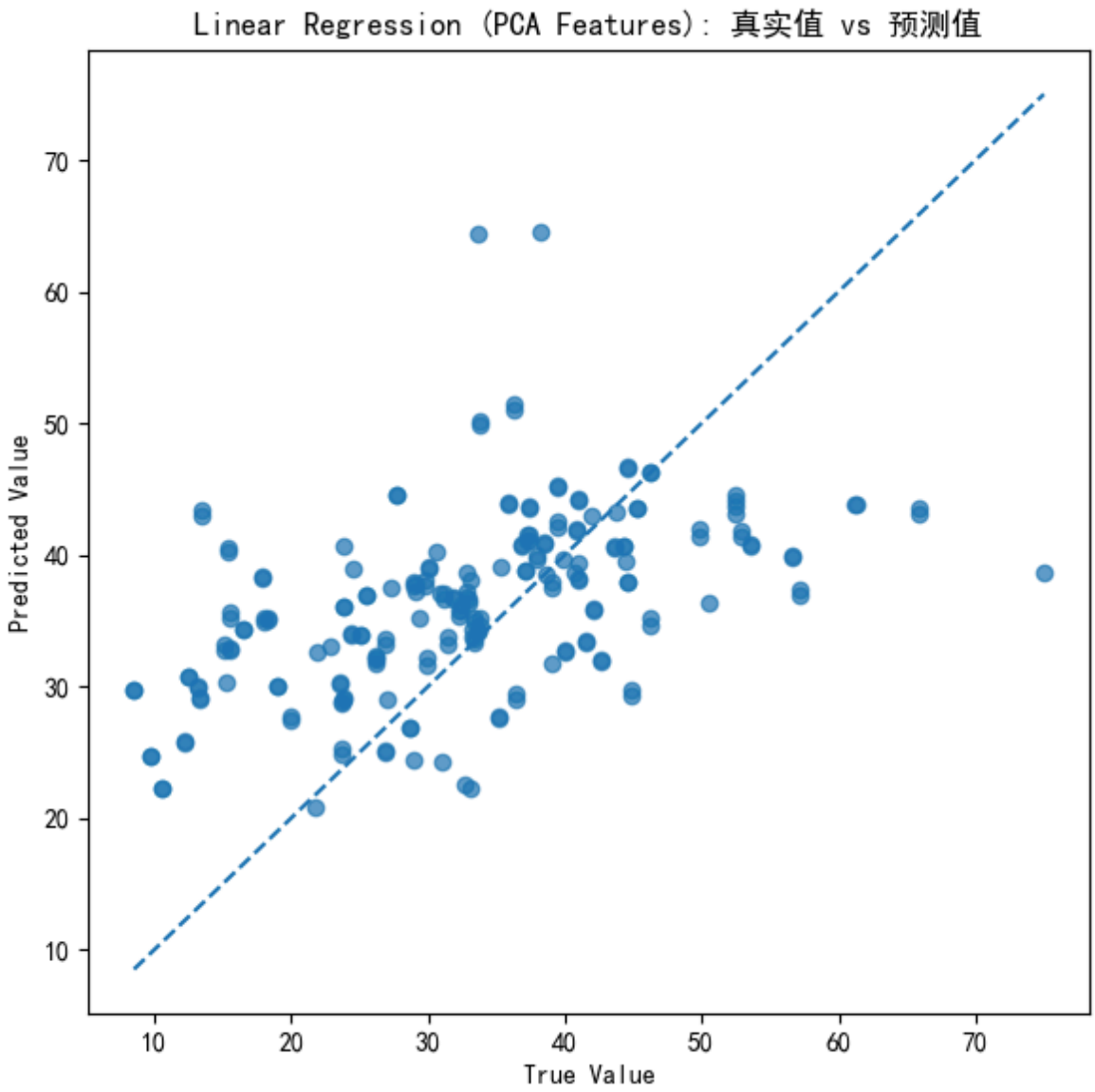

```
[Linear Regression (Selected Features)] 评估结果
MSE  = 131.9253
RMSE = 11.4859
MAE  = 8.8614
R²   = 0.1357
```






```
[Linear Regression (PCA Features)] 评估结果
MSE  = 128.0168
RMSE = 11.3145
MAE  = 8.8782
R²   = 0.1613
```



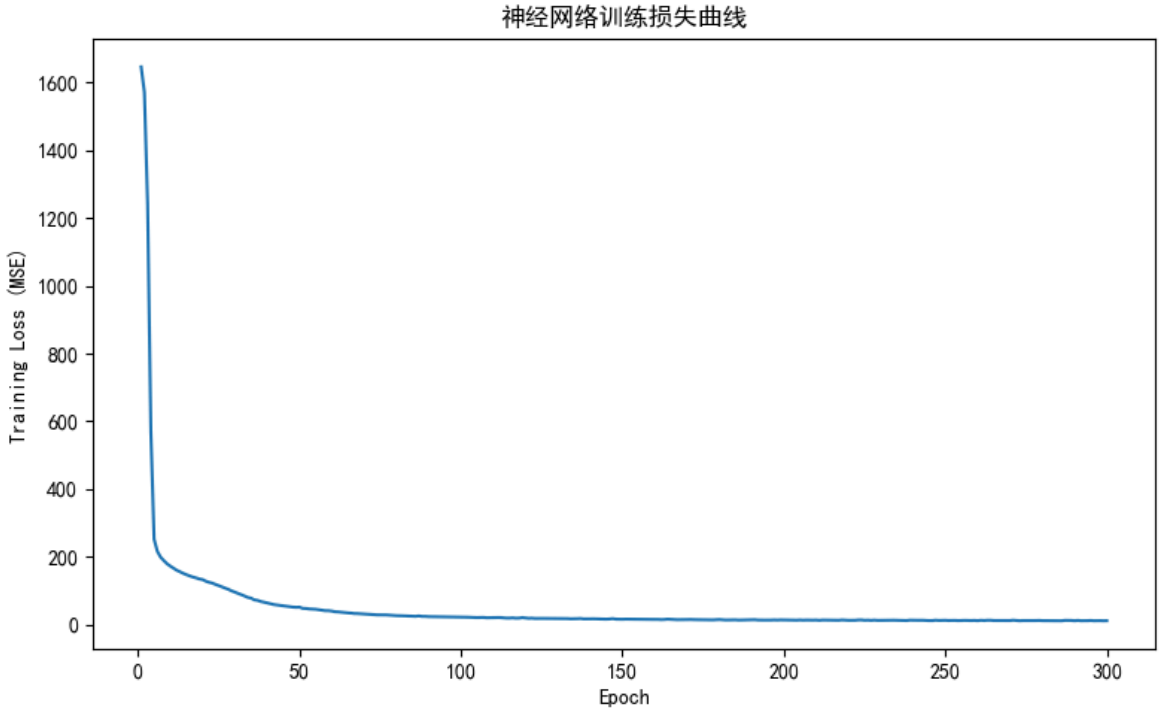


In [50]:
```
# 9. 神经网络模型（Pytorch）
```

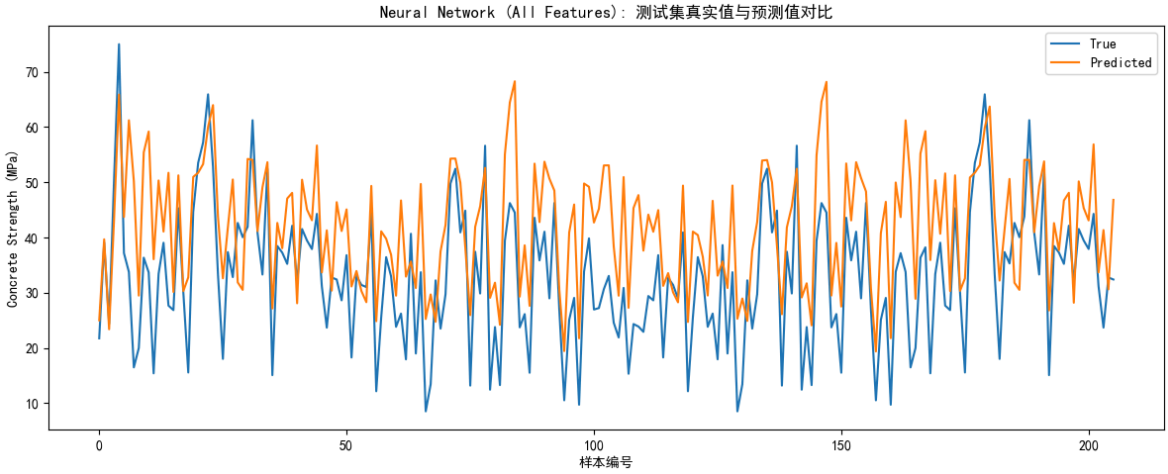
In [51]:
```
print("\n" + "=" * 80)
print("6) 神经网络回归")
print("=" * 80)

class MLPRegressor(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 1)
        )

    def forward(self, x):
        return self.net(x)


# 转为 Tensor
```

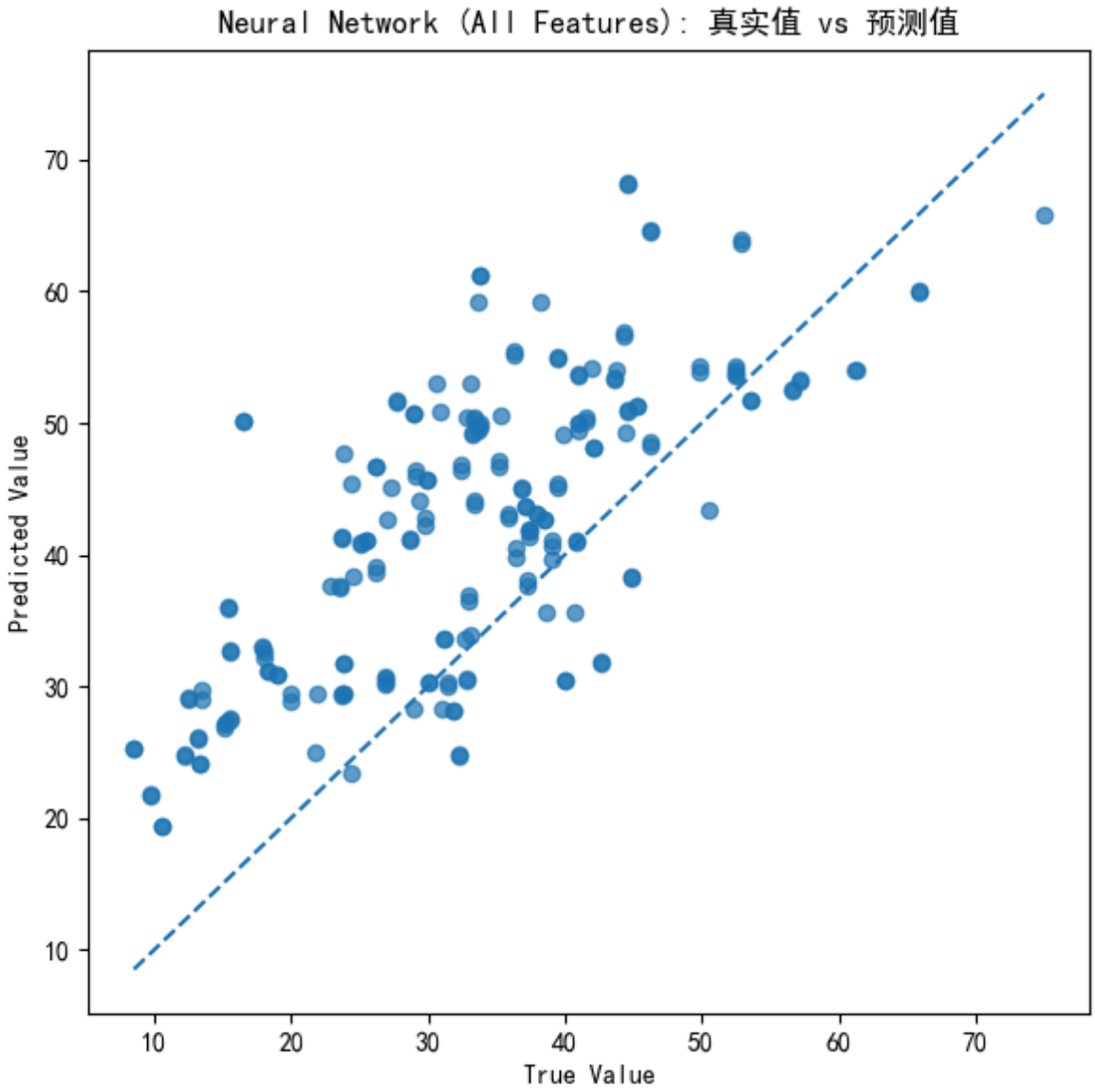
```
X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train.values.reshape(-1, 1), dtype=torch.float32
X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test.values.reshape(-1, 1), dtype=torch.float32)

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

model = MLPRegressor(input_dim=X_train.shape[1]).to(DEVICE)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

num_epochs = 300
train_losses = []

for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0.0

    for batch_X, batch_y in train_loader:
        batch_X = batch_X.to(DEVICE)
        batch_y = batch_y.to(DEVICE)

        optimizer.zero_grad()
        outputs = model(batch_X)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item() * batch_X.size(0)

    epoch_loss /= len(train_loader.dataset)
    train_losses.append(epoch_loss)

    if (epoch + 1) % 50 == 0:
        print(f"Epoch [{epoch + 1:>3}/{num_epochs}] - Train Loss: {epoch_loss:.4

# 损失曲线
plt.figure(figsize=(8, 5))
plt.plot(range(1, num_epochs + 1), train_losses)
plt.xlabel('Epoch')
plt.ylabel('Training Loss (MSE)')
plt.title('神经网络训练损失曲线')
plt.tight_layout()
plt.show()

# 测试集预测
model.eval()
with torch.no_grad():
    y_pred_nn = model(X_test_tensor.to(DEVICE)).cpu().numpy().flatten()

res_nn = evaluate_regression(y_test, y_pred_nn, "Neural Network (All Features)")
plot_prediction(y_test, y_pred_nn, "Neural Network (All Features)")
```

```
================================================================================
6) 神经网络回归
================================================================================
Epoch [ 50/300] - Train Loss: 51.3003
Epoch [100/300] - Train Loss: 21.7862
Epoch [150/300] - Train Loss: 15.5441
Epoch [200/300] - Train Loss: 13.4546
Epoch [250/300] - Train Loss: 12.4296
Epoch [300/300] - Train Loss: 11.3233
```




```
[Neural Network (All Features)] 评估结果
MSE  = 159.2575
RMSE = 12.6197
MAE  = 10.6057
R²   = -0.0434
```

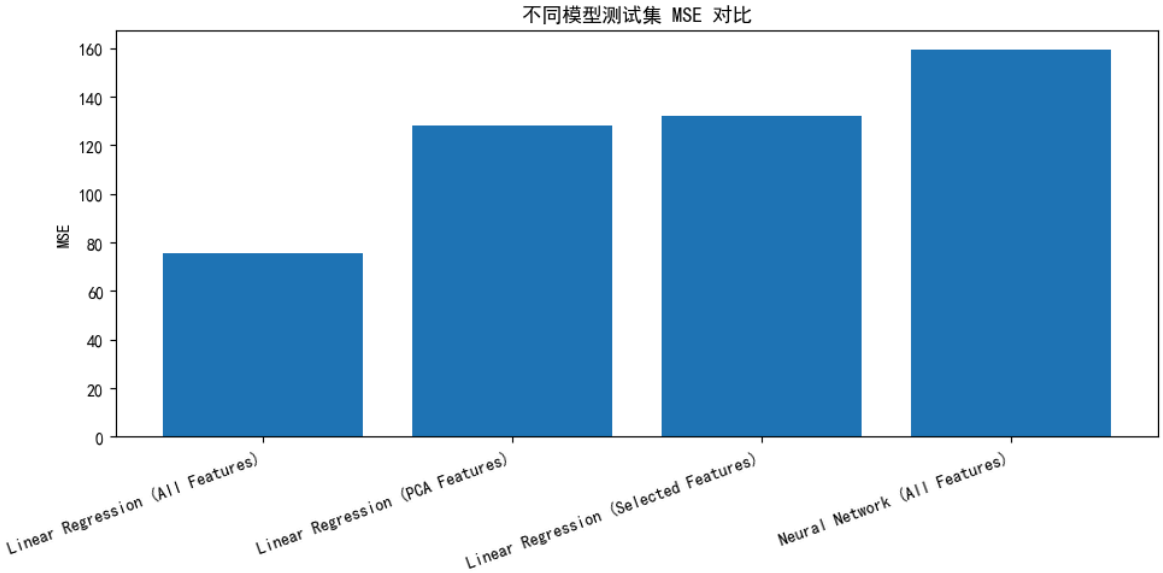




In [52]:
```
# 10. 结果汇总
```

In [53]:
```
results_df = pd.DataFrame([
    res_lr_all,
    res_lr_sel,
    res_lr_pca,
    res_nn,
]).sort_values(by='MSE', ascending=True).reset_index(drop=True)

print("\n" + "=" * 80)
print("7) 模型结果汇总（按 MSE 从小到大排序）")
print("=" * 80)
print(results_df)

# 柱状图比较各模型 MSE
plt.figure(figsize=(10, 5))
plt.bar(results_df['Model'], results_df['MSE'])
plt.xticks(rotation=20, ha='right')
plt.ylabel('MSE')
plt.title('不同模型测试集 MSE 对比')
plt.tight_layout()
plt.show()
```

```
================================================================================
7) 模型结果汇总（按 MSE 从小到大排序）
================================================================================
                                  Model         MSE       RMSE        MAE  \
0       Linear Regression (All Features)   75.743925   8.703099   6.872270
1       Linear Regression (PCA Features)  128.016819  11.314452   8.878183
2  Linear Regression (Selected Features)  131.925325  11.485875   8.861422
3          Neural Network (All Features)  159.257502  12.619727  10.605710

         R2
0  0.503742
1  0.161261
2  0.135653
3 -0.043421
```



In [54]:
```
# 11. 简要结论（自动输出）
```

In [55]:
```
best_model_name = results_df.loc[0, 'Model']
best_model_mse = results_df.loc[0, 'MSE']

print("\n" + "=" * 80)
print("8) 结论")
print("=" * 80)
print(f"在本次实验中，测试集上 MSE 最小的模型是：{best_model_name}")
print(f"其测试集 MSE 为：{best_model_mse:.4f}")
```

```
================================================================================
8) 结论
================================================================================
在本次实验中，测试集上 MSE 最小的模型是：Linear Regression (All Features)
其测试集 MSE 为：75.7439
```

综合实验结果可以发现，使用全部特征的线性回归模型在测试集上取得了最好的预测效果。这说明对于该数据集来说，简单的线性模型已经能够较好地捕捉变量之间的关系，而更复杂的模型不一定能够带来更好的性能。